

No Pain, Same Gain: Interference-Gated Aggregation for Constant-Memory Recurrence

Fábio de Oliveira Peres

Pesquisador Independente, Rio Verde, Goiás, Brasil

Preprint, junho de 2026

Resumo

O Transformer estabeleceu que a recorrência de estado fixo é dispensável para modelagem de sequências — atenção é tudo o que você precisa — mas a atenção paga por essa generalidade com um cache chave-valor que cresce com o contexto (memória $\propto T$, custo $O(T^2)$ por passo). RNNs de memória crescente [Behrouz et al., 2026] recuperam a recorrência armazenando uma memória associativa por segmento, de modo que a capacidade cresce com o comprimento, mas uma implementação fiel colapsa no multi-query associative recall (MQAR): 25,2% com 16 pares chave-valor contra 97,5% de uma memória de atenção linear *fixa* de mesma largura. Rastreamos o colapso até a *agregação*, não ao roteamento: normalizar o readout de cada segmento em cache antes da mistura destrói a supressão por denominador global que faz a atenção linear funcionar, de modo que qualquer gate não saturado mistura lixo em escala plena na magnitude do sinal. Propomos a **Agregação com Gating por Interferência (IGA)**, que aplica gate aos numeradores e denominadores por segmento *não-normalizados* dentro de uma única razão global; com gates inicializados em um, a IGA é numericamente idêntica à memória fixa (verificado até 5×10^{-7}), de modo que a memória fixa é um *piso* e o colapso é impossível por construção. A combinamos com um **roteador polinomial de tensor-sketch** que roteia por *identidade aprendida* sem ler token IDs — uma relaxação diferenciável de um count-min sketch — dando recall plano no comprimento de avaliação mesmo quando as queries nunca repetem chaves. Tornamos então o estado recorrente **constante no comprimento da sequência**: uma fatoração de posto $\leq S$ remove o termo d^2 exatamente, e limitar os segmentos vivos com um background comprimido de baixo posto limita o crescimento — 0,39 MB em $d = 64$ contra um cache KV que cresce $\propto T$, abaixo dele a partir de $T \approx 1k$ e $15\times$ menor projetado para um modelo de 1T parâmetros, sem perda de recall na tarefa principal. No benchmark próprio da literatura — o harness oficial Zoology MQAR, configuração-padrão mais difícil (512, 64) — a IGA atinge $99,2 \pm 0,1$ contra Mamba $23,2 \pm 14,0$ e Based $79,5 \pm 1,4$, empatando com a atenção softmax total (100,0). Em um modelo de linguagem real em nível de caractere (TinyStories, 2 camadas, $d = 128$, 3 seeds, dois contextos) a IGA empata com um baseline de memória fixa de mesma largura em bits-por-caractere dentro do ruído entre seeds — *memória constante sem custo de perplexidade* — enquanto uma análise de custo mostra que o prêmio único de treino é amortizado por economias de inferência por query que crescem com o contexto, já que a IGA é $O(T)$ por passo com estado constante onde a atenção softmax é $O(T^2)$ por passo com cache $\propto T$. **No pain, same gain**: o custo da memória crescente é removido, a qualidade é mantida.

1 Introdução

“Atenção é tudo o que você precisa” [Vaswani et al., 2017] aposentou a recorrência de estado fixo para modelagem de sequências, mas o preço está escrito no mecanismo: um cache chave-valor que guarda cada token passado. O cache cresce linearmente com o contexto e o produto de atenção

por passo é quadrático, de modo que memória e computação escalam com a sequência. Essa é a *dor* do paradigma de atenção — suportável em contexto curto, dominante em contexto longo, e eventualmente a restrição determinante.

RNNs de memória crescente [Behrouz et al., 2026] oferecem a aposta oposta: mantenha um estado recorrente, mas deixe-o *crescer* armazenando uma memória associativa por segmento de S tokens e agregando os readouts em cache com um gate aprendido. A capacidade então escala com o comprimento enquanto o custo por token permanece sub-quadrático. A promessa é o recall da atenção com a estrutura de custo da recorrência. Verificamos que uma implementação fiel de camada única dessa promessa não apenas falha em superar a memória fixa que ela generaliza — ela colapsa bem abaixo dela (Seção 4, Tabela 1).

Este paper faz uma única proposta afiada: um modelo de memória crescente que carrega **no pain, same gain** — sem custo de memória crescente, e a mesma qualidade dos baselines fortes. Entregamos em três passos. Primeiro, consertamos a agregação: o colapso é causado pela normalização do readout por segmento, e aplicar gate ao estado por segmento *não-normalizado* dentro de uma razão global única (IGA) o remove, com a memória fixa como um piso de inicialização demonstrável. Segundo, tornamos o gargalo restante — roteamento de segmento — solucionável sem token IDs, via um roteador polinomial de tensor-sketch que roteia por identidade aprendida. Terceiro, tornamos o estado *constante no comprimento da sequência*: uma fatoração de posto $\leq S$ remove o termo quadrático-na-largura exatamente, e limitar os segmentos vivos com um background comprimido de baixo posto limita o crescimento, dando um estado sub-KV constante sem perda de recall na tarefa principal. O slogan nomeia exatamente o que as medições sustentam: *no pain* — a memória crescente e o passo $O(T^2)$ sumiram, deixando um estado constante e computação $O(T)$ por passo; *same gain* — o recall iguala a atenção total na tarefa-padrão mais difícil da literatura (99,2 vs 100,0) e o bits-por-caractere iguala o baseline de memória fixa em texto real (dentro do ruído entre seeds). O título é uma compressão do resultado, não um slogan.

Contribuições.

1. **Diagnóstico e conserto da agregação (§3.1).** O colapso da memória crescente é uma falha de *agregação*: a normalização por segmento apaga a diferença de massa de match entre segmentos. A IGA aplica gate a pares numerador/denominador não-normalizados dentro de uma razão global única; com $\text{gate} \approx 1$ (init) o modelo é a memória fixa até 5×10^{-7} , de modo que o aprendizado só pode *remover* a interferência entre segmentos — a fonte da capacidade que faz K memórias pequenas vencerem uma grande.
2. **Roteamento por identidade aprendida (§3.2).** O roteamento por similaridade somada é limitado pela família de kernels: ele acompanha a saturação da memória fixa através de famílias de scores e mesmo sob supervisão auxiliar, enquanto resumos que preservam identidade roteiam exatamente mas exploram repetição literal de chaves. Nosso roteador polinomial de tensor-sketch [Pham and Pagh, 2013] é uma relaxação diferenciável, sem IDs, de um count-min sketch que preserva a propriedade de recall plano, e permanece plano no **MQAR-indirect**, um benchmark onde as queries nunca repetem chaves — recuperado sem rótulos pela destilação de um professor quadrático treinado apenas na loss de recall.
3. **Estado de memória constante (§3.3).** Uma fatoração de posto $\leq S$ do cache por segmento remove o termo d^2 exatamente ($< 5 \times 10^{-7}$); limitar os segmentos vivos em $K_{\max}$ com um background comprimido de baixo posto torna o estado recorrente constante em T — 0,39 MB em $d = 64$, abaixo do cache KV a partir de $T \approx 1\text{k}$ e $15\times$ menor projetado para 1T parâmetros, sem perda de recall na tarefa (512, 64).

4. **Validação em texto real e custo de deploy (§5–§6).** Em um modelo de linguagem real em nível de caractere (3 seeds, dois contextos) a IGA empata com um baseline de memória fixa de mesma largura em bits-por-caractere dentro do ruído entre seeds — o estado de memória constante não carrega penalidade de perplexidade — enquanto a vantagem de recall associativo mostra-se específica da tarefa (não transfere para um probe de fato plantado em texto natural nesta escala). Uma análise de custo mostra que o prêmio único de treino (o gate) é amortizado por economias de inferência por query que crescem com o contexto, já que a IGA é $O(T)$ por passo com estado constante contra os $O(T^2)$ por passo da atenção softmax com cache $\propto T$.

Posicionamos os claims de recall em pequena escala (camada única, $d = 64$, recall sintético) e comparamos contra atenção exata, Based [Arora et al., 2024] e Mamba [Gu and Dao, 2023] sob o protocolo oficial Zoology [Arora et al., 2023] com 3 seeds em toda parte; o experimento de modelo de linguagem (§5) é a única configuração em texto real.

2 Contexto e setup

MQAR. A entrada é `k1 v1 ... kN vN SEP kq1 vq1 ... kqQ vqQ`; o modelo deve prever cada valor consultado como o próximo token após sua chave de query. Usamos a formulação estilo Zoology com N até 128 pares, $Q = 4$ queries, vocabulário de chaves 192, vocabulário de valores 16 (acaso 6,25%); treino sempre em $N = 48$, 4000 passos, batch 256, AdamW 3×10^{-4} , seed idêntica antes de cada modelo; avaliação em $N \in \{48, 64, 96, 128\}$ sem retreino. No §4 usamos adicionalmente o gerador e protocolo *oficiais* do Zoology (vocabulário 8192, lacunas de query em lei de potência, varredura de learning rate sobre logspace $(-4, -2, 4)$, weight decay 0,1, schedule cosseno, parada precoce em 99%).

Memória fixa (FIXA). Atenção linear causal [Katharopoulos et al., 2020] com feature map $\varphi(x) = \text{elu}(x) + 1$:

$$y_t = \frac{\sum_{j \leq t} \varphi(q_t) \cdot \varphi(k_j) v_j}{\sum_{j \leq t} \varphi(q_t) \cdot \varphi(k_j)}.$$

Uma convolução causal depthwise curta ($k = 3$) antes das projeções permite que uma camada ligue pares chave–valor, como no Based/Mamba. A capacidade é limitada por d^2 .

Memória crescente (GRM). Por segmento i : $M^{(i)} = \sum_j v_j \varphi(k_j)^\top$, $z^{(i)} = \sum_j \varphi(k_j)$. Na query t , cada segmento em cache contribui um readout *normalizado* $r_i = M^{(i)} q_t / (z^{(i)} \cdot q_t)$, misturado por um softmax sobre massas de match. Esse é o mecanismo que colapsa.

3 Método: Agregação com Gating por Interferência

3.1 Agregação que não pode colapsar

A memória fixa é *exatamente* a agregação gate- $\equiv 1$ dos caches por segmento não-normalizados:

$$y_{\text{fix}} = \frac{\sum_i M^{(i)} q}{\sum_i z^{(i)} \cdot q}.$$

A IGA insere o gate dentro dessa razão:

$$y_t = \frac{N_{\text{cur}} + \sum_{i < s} g_i M^{(i)} q_t}{m_{\text{cur}} + \sum_{i < s} g_i z^{(i)} \cdot q_t}, \quad g_i = \sigma(a(\log s_i - \text{ref}) + b),$$

onde s_i é um score de roteamento e ref é a média (scores de similaridade) ou o máximo (scores de contagem) de $\log s_i$. Com $b = 3$, $g \approx 0,95$ em todo lugar na init $\Rightarrow$ o modelo *começa* como a memória fixa; o teste de unidade `test_ungated_equals_fixed` verifica a identidade até 5×10^{-7} . O gate só pode *remover* interferência — que é precisamente de onde vem a capacidade de K memórias pequenas sobre uma grande.

Por que o GRM colapsa. Instrumentando o GRM treinado em $n = 16$: recall 25,2%, acurácia de roteamento do seu gate softmax 90,7%, massa do gate no segmento atual ≈ 0 . O matador é que readouts normalizados de segmentos errados são lixo *na mesma magnitude do sinal*; um softmax 0,7/0,3 mistura 30% de lixo em escala plena. Nenhum conserto no softmax ajuda; a normalização tem que se mover *para fora* da mistura — o que a IGA faz.

3.2 Roteamento por identidade aprendida

Com a agregação consertada, a lacuna residual da IGA ao oráculo é roteamento puro. O problema de roteamento é um problema da *família de kernels*: todo score de similaridade somada (primeiro/segundo momento, features elementwise-exponenciais, embeddings aprendidas com supervisão auxiliar) satura exatamente como a memória fixa, enquanto resumos que *preservam identidade* — o histograma de tokens ou seu count-min sketch [Cormode and Muthukrishnan, 2005] — roteiam exatamente (100%/93%+) com zero parâmetros e dão recall plano no comprimento de avaliação, mas eles leem token IDs e as queries do MQAR repetem chaves literalmente. Linguagem real não.

Precisamos de um score $s_i(q) \approx$ (ocorrências da identidade de q no segmento i), armazenável como uma soma por segmento, diferenciável, e sem IDs. Para embeddings quase ortonormais $\hat{e}$, $(\hat{e}_q \cdot \hat{e}_k)^p \rightarrow \delta(q = k)$ quando p cresce, e o kernel polinomial de grau p é exatamente linearizável: $(x \cdot y)^p = \langle x^{\otimes p}, y^{\otimes p} \rangle$, compressível por tensor sketch [Pham and Pagh, 2013] com $\mathbb{E}[\text{ts}(x) \cdot \text{ts}(y)] = (x \cdot y)^p$ e variância $O(1/m)$ para vetores unitários — custo *polinomial* na nitidez, onde estimadores estilo Performer de um $\exp(\tau q \cdot k)$ comparavelmente nítido requerem variância exponencial em τ [Choromanski et al., 2021]. O score do roteador é

$$s_i = \text{softplus}(w_1) (\hat{e}_q \cdot \sum_j \hat{e}_{k_j}) + \text{softplus}(w_p) \text{med}_J(\text{ts}(\hat{e}_q) \cdot \sum_j \text{ts}(\hat{e}_{k_j})),$$

com três elementos de design: (i) a **mediana de $J = 3$ sketches independentes** — o análogo contínuo do min do count-min — eleva o roteamento de 96,6% para 99,8% num microbenchmark sem treino nas estatísticas exatas de segmento do MQAR ($p = 4$, $m = 256$, $r = 64$), onde grau-1 sozinho rende apenas 39,9–60,8% (a mesma saturação que limita a memória fixa); (ii) um **termo de grau 1 para cold start**, já que $\partial(q \cdot k)^p / \partial \text{emb}$ se anula na ortogonalidade para $p \geq 2$, dando gradientes de alinhamento estilo word2vec quando os embeddings devem ser *aprendidos*, com pesos de mistura aprendíveis que deixam o treino anula-lo; (iii) **ligações de gate contagem-vs-similaridade** — scores de contagem fazem gate em $\log(\text{clamp}(s))$ com referência máxima (supressão forte desde a init), enquanto scores de poly aprendido fazem gate em $\log(\text{softplus}(s))$ e supervisionam scores crus como logits, porque scores aprendidos podem ser negativos no cold start e $\log(\text{clamp})$ cria uma zona de gradiente morto que destrói o treino do readout (essa distinção custou uma run completa falha, documentada como resultado negativo). Custo do resumo: $r + J \cdot m = 832$ floats/segmento (count-min: 256; histograma: V).

MQAR-indirect: a identidade tem que ser aprendida. Introduzimos um benchmark no qual o bloco de query usa um vocabulário de query *disjunto*, pareado às chaves por uma bijeção fixa ao longo do dataset — estrutura de sequência, posições e protocolo inalterados. Por construção,

roteadores por contagem de IDs recebem sinal zero. Dois regimes de treino: (a) cross-entropy auxiliar de roteamento com rótulos de bijeção — *supervisão privilegiada*, um limite superior para a família de kernels; (b) *destilação* — um professor mínimo de atenção softmax quadrática ($d = 64$, 1 camada, embeddings próprios) treinado apenas na loss de recall, cuja massa de atenção por segmento é destilada no roteador por KL; o professor é descartado na inferência (custo quadrático só durante o treino, espelhando SMT [Kumar and Isola, 2026]). O professor resolve a tarefa em 100%, confirmando que a atenção softmax não sofre do equilíbrio perverso.

3.3 Estado de memória constante

Uma memória crescente que supera a memória fixa em recall mas cresce sem limite ainda não responde à questão do paradigma — seu estado eventualmente excederia o cache KV que deve substituir. Tornamos o estado recorrente **constante em T** em dois passos, ambos apresentados como design e não como remendo.

Estado fatorado. O cache por segmento $M^{(i)} = \sum_{j \in \text{seg}_i} v_j \varphi(k_j)^\top$ é uma soma de S atualizações de posto 1, de modo que $\text{rank}(M^{(i)}) \leq S = 16$. Fatoramo-lo exatamente como $M^{(i)} = U^{(i)\top} W^{(i)}$ com $U^{(i)}, W^{(i)} \in \mathbb{R}^{S \times d}$ (posto $R = S$), armazenando $2dS$ floats por segmento em vez de d^2 e computando o readout $M^{(i)}q = U^{(i)\top} (W^{(i)}q)$ a custo $2dS$ em vez de d^2 . A fatoração é exata para uma matriz de posto $\leq S$, de modo que a saída do modelo é inalterada até erro de ponto flutuante — `test_factored_equals_dense` verifica a identidade (limiar 10^{-4} ; lacuna medida $< 5 \times 10^{-7}$) — e na tarefa oficial do Zoology (512, 64) o modelo fatorado iguala o denso dentro do ruído entre seeds (denso $99,02 \pm 0,17$, fatorado $99,18 \pm 0,08$, 3 seeds). A fatoração remove o termo quadrático-na-largura ($d^2 \rightarrow 2dS$: 4096 $\rightarrow$ 2048 elementos em $d = 64$), mas o estado ainda cresce com o número de segmentos, $O(T/S)$.

Memória limitada (capped). Limitamos o número de segmentos em cache *vivos* em $K_{\max}$ e comprimimos o overflow num único background de baixo posto de posto r_{bg} . O estado recorrente se torna

$$\text{state} = K_{\max} \cdot \text{per_seg} + (2d r_{\text{bg}} + d + m) + K_{\max},$$

independente de T , onde $\text{per_seg} = 2dS + d + m$. A compressão é uma evacuação FIFO determinística com normalização do readout comprimido; segmentos vivos e o background comprimido são misturados dentro da razão global única da IGA, de modo que a agregação com gating por interferência do §3.1 é preservada. Na tarefa oficial (512, 64) o contexto é exatamente $K_{\max} = 32$ segmentos, de modo que nenhuma evacuação ocorre e o modelo limitado é um no-op: o recall é idêntico ao modelo fatorado não-limitado — 99,23/99,30/99,02 (média 99,18) em três seeds. O preço honesto da memória constante é que segmentos mais antigos que $K_{\max}$ são comprimidos (com perdas): o recall de fatos dentro dos últimos $K_{\max}$ segmentos acompanha o baseline ilimitado mesmo com centenas de evacuações, enquanto fatos mais antigos não são individualmente recuperáveis — o tradeoff explícito recall/memória.

4 Experimentos — recall associativo

Todos os números são média $\pm$ desvio em 3 seeds salvo nota. Baselines: atenção exata (MHA de 2 camadas com posições aprendidas), Based (híbrido oficial: BaseConv + atenção linear de Taylor), Mamba (S6; reimplementação em PyTorch puro com scan paralelo exato, testado por unidade contra

a referência sequencial). Backbone, inicialização (esquema GPT-2) e hiperparâmetros seguem a configuração Figure-2 do Zoology. Um achado negativo, verificado contra a implementação *oficial*: a configuração Figure-2 de atenção apenas de 2 camadas não forma o circuito de indução sob nosso protocolo de fluxo infinito (amostras frescas a cada passo removem o bootstrap de memorização); ela estabiliza em $\sim 13\%$ através de learning rates, e o `zoology.model.LanguageModel` oficial treinado sob o mesmo loop dá 12,3% — idêntico à nossa réplica — enquanto sob o regime oficial de dados fixos nossa réplica *reproduz* a transição de induction-head publicada. Reportamos portanto **attention-conv** — um modelo de atenção exata de camada única com o mesmo stem de convolução curta que todo outro modelo tem — como a âncora superior funcional.

4.1 MQAR literal (nosso protocolo)

Treino em $n = 48$ ($n_k = 192$), avaliação sem retreino; acaso 6,25%.

modelo	$n=48$	$n=64$	$n=96$	$n=128$
atenção (Zoology fig2)	12.9 ± 0.0	11.1 ± 0.5	9.7 ± 0.5	9.0 ± 0.4
based	8.3 ± 0.1	7.9 ± 0.2	7.3 ± 0.2	7.3 ± 0.1
mamba	13.3 ± 0.0	12.3 ± 0.1	10.6 ± 0.2	9.6 ± 0.4
FIXA(64)	77.9 ± 1.0	71.6 ± 0.8	59.9 ± 0.4	50.9 ± 0.4
FIXA(128)	92.1 ± 0.5	88.8 ± 0.6	81.8 ± 1.0	74.5 ± 1.1
IGA-hist (IDs)	98.3 ± 0.1	98.3 ± 0.1	98.4 ± 0.2	98.5 ± 0.1
IGA-sketch (IDs)	98.1 ± 0.1	98.1 ± 0.1	98.1 ± 0.2	98.0 ± 0.2
IGA-poly (sem IDs)	97.7 ± 0.1	97.6 ± 0.2	97.5 ± 0.2	97.1 ± 0.2

Todas as variantes IGA são planas no comprimento de avaliação; o roteador polinomial sem IDs paga 0,8–1,4 pontos de ruído de sketch relativo ao histograma exato. Os baselines de literatura de 2 camadas falham nesse regime (48 pares em $d = 64$, muito além do cenário favorável à capacidade deles).

4.2 MQAR-indirect

Mesmo protocolo; a query nunca repete a chave.

modelo	$n=48$	$n=64$	$n=96$	$n=128$
atenção (Zoology fig2)	13.1 ± 0.1	11.5 ± 0.4	9.9 ± 0.2	9.2 ± 0.2
based	7.4 ± 0.3	7.3 ± 0.1	6.8 ± 0.2	6.8 ± 0.1
mamba	13.3 ± 0.0	12.2 ± 0.1	10.6 ± 0.3	9.6 ± 0.3
FIXA(64)	57.6 ± 20.8	51.4 ± 19.4	42.5 ± 16.2	36.8 ± 13.9
IGA-sketch (controle)	57.2 ± 20.6	50.9 ± 19.1	42.1 ± 15.9	36.4 ± 13.6
IGA-emb+aux (fam. exp)	68.6 ± 17.8	62.2 ± 17.9	52.1 ± 16.0	45.1 ± 13.8
IGA-poly+aux (privilegiado)	96.1 ± 0.7	95.8 ± 0.7	95.4 ± 0.7	94.8 ± 0.9
IGA-poly destilado (sem rótulos)	93.5 ± 1.0	92.6 ± 1.4	90.5 ± 1.8	88.2 ± 2.2

O controle por contagem de IDs acompanha a FIXA seed a seed (roteamento 6,0% = acaso), confirmando que o benchmark remove o atalho literal por construção. O roteador de kernel polinomial preserva a propriedade de recall plano com identidade *totalmente aprendida*: +58 pontos sobre a memória fixa em $n = 128$, e a variante destilada sem rótulos retém 93–96% do limite superior privilegiado.

4.3 Protocolo oficial Zoology (vocabulário 8192)

Gerador e loop de treino oficiais; 3 seeds, média $\pm$ desvio.

modelo	(128, 8) fácil	(512, 64) difícil
atenção (softmax total)	99.4 ± 0.3	100.0 ± 0.0
attention-conv (âncora 1-camada)	99.3 ± 0.3	99.4 ± 0.0
based	99.2 ± 0.2	79.5 ± 1.4
mamba	99.1 ± 0.0	23.2 ± 14.0
IGA-sketch (nosso)	99.1 ± 0.0	99.1 ± 0.0
IGA-poly (nosso, ident. aprendida)	99.1 ± 0.0	99.2 ± 0.1

O regime discriminante é (512, 64): 64 pares chave-valor sobre um contexto de 512 tokens (32 segmentos de $S = 16$ — $2\times$ mais segmentos que qualquer cenário acima). Todos os seis modelos resolvem a tarefa fácil. Na tarefa difícil os baselines eficientes se separam exatamente como o paper Based relata: **Mamba colapsa** ($23,2 \pm 14,0$, uma seed em 7% — a saturação de estado fixo) e **Based degrada** ($79,5 \pm 1,4$). **Ambos os modelos IGA se mantêm em $\sim 99\%$** , empatando com a atenção softmax total (100%) e a âncora de atenção convolucional (99,4), a custo por token sub-quadrático. Esse é o resultado citável que fecha a comparação: no benchmark próprio da literatura, na sua configuração-padrão mais difícil, a memória crescente mantém recall quase perfeito onde os modelos de estado fixo falham.

4.4 Custo de inferência

Batch 1, $d = 64$, fp32, loops Python de referência em ambos os lados (comparação de escala, não constantes de engenharia).

modelo	$T=4096$ tok/s	$T=16384$ tok/s	$T=65536$ tok/s	estado @ $T=65536$
atenção (SDPA, cache KV)	3577	997	253	32.0 MB ($\propto T$)
IGA-poly recorrente, gate soft	3337	3721	3167	0.39 MB (const)
IGA-poly recorrente, top-1 duro	4885	4725	4455	0.39 MB (const)

A vazão da IGA é $\sim$ plana no comprimento do contexto; a atenção exata incremental degrada linearmente, $17,6\times$ mais lenta em 64k. A coluna de estado relata o estado de memória constante do §3.3 — 0,39 MB constante — contra um cache KV que cresce $\propto T$ (32 MB em 64k). O crossover de vazão e o estado sub-KV constante são os claims.

4.5 Escalonamento de memória constante e o regime de contexto longo

Com o estado fatorado + limitado, o estado recorrente é constante em T . Medido com `state_bytes` ($d = 64$, $K_{\max} = 32$, $r_{\text{bg}} = 16$, fp32) em 0,39 MB contra o cache KV $2Td$:

O estado iguala o cache no crossover $T \approx 2K_{\max}S$ (onde o cache enche exatamente $K_{\max}$ segmentos); abaixo dele o estado constante é maior — ele é dimensionado para capacidade, não para o contexto atual — e acima dele a vantagem cresce sem limite. Projetado para um regime de 1T parâmetros ($d = 8192$, $K_{\max} = 4096$, $r_{\text{bg}} = 4$), o estado constante é 209,5 GB = $0,065\times$ o cache KV de 3,1 TB — $15\times$ **menor, e constante** — onde o estado denso da IGA seria 800 TB ($256\times$ maior)

T	cache KV	IGA (constante)	razão
512	0.25 MB	0.39 MB	$1.53\times$
1024	0.50 MB	0.39 MB	$0.76\times$
2048	1.00 MB	0.39 MB	$0.38\times$
4096	2.00 MB	0.39 MB	$0.19\times$
65536	32.0 MB	0.39 MB	$0.012\times$

e mesmo o estado fatorado-mas-ilimitado é 3,2 TB (paridade, ainda crescente). O estado sub-KV constante é o resultado de memória.

5 Modelagem de linguagem em texto real: memória constante sem custo de perplexidade

Todos os resultados do §4 usam recall associativo sintético. Fechamos a questão restante — o IGA de memória constante carrega uma penalidade de perplexidade em *texto real*? — treinando um modelo de linguagem IGA de 2 camadas, $d = 128$ (tamanho de segmento 64; roteador polinomial de tensor-sketch lendo tokens crus; estado fatorado do §3.3) contra a arquitetura *idêntica* com o gate desativado (`gated=False`, o baseline de memória fixa FIXA de mesma largura e contagem de parâmetros). Os dados são TinyStories em nível de caractere, 88 MB, baixados — nenhum dado sintético é gerado para qualquer experimento de modelo de linguagem. Otimizador AdamW 3×10^{-4} , batch 16, 4000 passos, 3 seeds, contextos {1024, 2048}. Reportamos bits-por-caractere (bpc) numa fração held-out de 5% e um probe de recall de fato plantado — o análogo do MQAR em texto natural — em três distâncias (ctx/4, ctx/2, ctx-64).

ctx	IGA bpc	FIXA bpc	Δ bpc (IGA-FIXA)	recall@ctx-64 IGA	recall@ctx-64 FIXA
1024	2.1607 ± 0.0060	2.1621 ± 0.0084	-0.0015	1.67	1.67
2048	2.1138 ± 0.0114	2.1143 ± 0.0127	-0.0005	2.29	2.08

Dois achados. (i) **Sem custo de perplexidade.** A diferença de bpc é -0,0015 (ctx 1024) e -0,0005 (ctx 2048), ambas uma ordem de magnitude abaixo do desvio por seed (0,006-0,013); a IGA nunca é consistentemente pior nas seis runs (modelo, seed). O estado de memória constante portanto não degrada a modelagem de linguagem relativamente a um modelo de memória fixa de mesma largura e contagem de parâmetros. (ii) **A vantagem de recall é específica da tarefa.** O probe de fato plantado é próximo do acaso para ambos os modelos em ambos os contextos (1,7-2,3%, spread por seed 0-4,4% — ruído puro, sem dependência de distância); a vantagem de recall plano do §4 não transfere para a recuperação de um fato plantado arbitrário em texto natural em $d = 128$. O benchmark estruturado MQAR permanece o instrumento certo para o claim de recall associativo; a recuperação em texto natural nesta escala não separa as arquiteturas, e escalar d/ctx para surgir uma separação fica como trabalho futuro (limitado por orçamento de GPU).

Computação. A IGA custa $5,2\times$ (ctx 1024) e $4,9\times$ (ctx 2048) mais wall-clock por passo que a FIXA na nossa implementação de referência em loop Python — a normalização por segmento do gate, dominada por overhead de loop em vez de FLOPs assintóticos. O tradeoff é memória-vs-computação (estado $O(1)$ constante contra computação por passo maior), não memória-vs-qualidade: a qualidade é igualada dentro do ruído.

Notamos honestamente que uma run preliminar de 300 passos no projeto-pai reportou um ganho de $\sim 0,016$ bpc em $\text{ctx} \geq 2048$; ele **não** reproduz sob o protocolo de 3 seeds, 4000 passos aqui ($\Delta = -0,0005$, dentro do ruído). O resultado de modelo de linguagem é um resultado de não-regressão, não um ganho de qualidade.

6 Economia de deploy: onde a memória constante compensa

Esta seção sintetiza o §4.4 (vazão), §4.5 (memória) e §5 (qualidade LM) no quadro de custo de deploy — o regime em que um modelo de memória crescente é de fato destinado a substituir um transformer com cache KV.

Treino (única vez). Por passo, a atenção softmax é $O(T^2d)$ — os produtos $QK^\top$ e AV sobre a matriz $T \times T$ completa — enquanto a recorrência de atenção linear da IGA é $O(Td^2)$ (a atualização do estado fatorado é $O(TdS)$, menor). O escalonamento assintótico de FLOPs portanto favorece a IGA conforme o contexto cresce, com razão T/d : $\sim 8\times$ menos FLOPs de atenção em $\text{ctx } 1024$ ($d = 128$), $\sim 512\times$ em $T = 64k$ ($d = 64$). Não treinamos um baseline transformer de modelo de linguagem, de modo que declaramos a comparação de custo de treino como análise de complexidade, não como número medido; a única comparação que *medimos* ponta a ponta é a vazão de inferência contra atenção exata (§4.4: $14\text{--}17,6\times$ em $64k$). O prêmio de wall-clock de $5\times$ do §5, em contraste, é medido — contra o *baseline linear de memória fixa* (FIXA), o controle same-family relevante para o overhead do gate.

Inferência (recorrente, por token gerado). Um transformer com cache KV lê o cache completo por token — tráfego de memória $O(TdL)$ — e mantém um cache de $2TdL$ que cresce $\propto T$. A IGA lê um $O(K_{\max}dS)$ constante por token e mantém um estado constante. Medido em batch 1, $d = 64$ (§4.4): a IGA mantém $\sim 4,5k$ tok/s plano de $T = 4k$ a $64k$, enquanto a atenção exata incremental degrada $\sim 14\times$ para 253 tok/s em $64k$. O custo de inferência por token da IGA é constante em T ; o do transformer cresce com T .

Amortização. O treino é pago uma vez; a inferência é paga por query ao longo da vida de serving do modelo. O prêmio único de $5\times$ do gate (contra o baseline linear) é amortizado pelas economias recorrentes de inferência uma vez que o contexto servido é longo o bastante para o estado constante ficar abaixo do cache — o crossover $T \approx 2K_{\max}S$ do §4.5, $15\times$ menor projetado para $1T$. Abaixo do crossover, onde o cache é ele próprio pequeno e a atenção é rápida, o transformer é a opção mais barata e a vantagem da IGA é apenas a capacidade de recall do §4; acima dele, o custo por token da IGA para de crescer enquanto o do transformer continua crescendo, de modo que o custo total de servir contextos longos eventualmente favorece a IGA por uma margem ilimitada. Passado o ponto em que o cache KV não cabe mais no dispositivo, a IGA não é a opção mais barata mas a única viável.

Esse é um argumento de estrutura de custo, não uma cifra de deploy em dólar: o custo absoluto depende de hardware, batching e fusão de kernels não modelados aqui (loops Python de referência). O claim estrutural — memória $O(1)$ e computação por token $O(1)$ -em- T contra $O(T)$ e $O(T)$ para o cache, com qualidade LM igualada (§5) — é o que o resultado de memória constante compra. Esse é o “no pain”: o custo da memória crescente é removido. O “same gain” é o recall igualado (§4.3) e a perplexidade igualada (§5).

7 Trabalho relacionado

Benchmarks de recall em atenção eficiente. Zoology [Arora et al., 2023] estabeleceu o MQAR como a tarefa discriminante entre atenção e misturadores eficientes; Based [Arora et al., 2024] enquadra o tradeoff recall-vazão. Adotamos o harness deles e adicionamos MQAR-indirect, que isola o *aprendizado de identidade* da *exploração de identidade* — ao nosso conhecimento o primeiro benchmark de recall onde as queries nunca repetem chaves por construção.

Memória crescente. Memory Caching [Behrouz et al., 2026] propõe memórias em cache por segmento com agregação gated (GRM entre suas variantes). Mantemos seu layout de estado $(M^{(i)}, z^{(i)})$ e custo, e substituímos sua agregação (a causa do colapso) e seu roteamento (a causa da saturação). Seu estado cresce com o número de segmentos em cache ($O(T/S)$, quadrático na largura via d^2); nosso estado fatorado + limitado (§3.3) é constante em T e linear na largura, removendo o crescimento que de outra forma excederia o cache KV que deve substituir.

Sketches polinomiais em atenção. PolySketchFormer [Kacham et al., 2023] usa sketches polinomiais para *substituir o kernel de atenção em si*. Nosso uso é diferente: o sketch resume a *identidade do segmento* para roteamento numa RNN de memória crescente; atenção/readout mantém seu próprio feature map. Count sketch [Charikar et al., 2002], count-min [Cormode and Muthukrishnan, 2005] e tensor sketch [Pham and Pagh, 2013] fornecem os estimadores; a variante mediana-de- J espelha o min do count-min sob estimativas com sinal.

Atenção linear e SSMS. A memória fixa é Katharopoulos et al. [2020]; Based e Mamba são os baselines de recall em pequena escala mais fortes nessa família. Performer [Choromanski et al., 2021] aproxima $\exp(q \cdot k)$ com features aleatórias; nosso argumento de variância (§3.2) explica por que essa família não pode fornecer roteamento de identidade δ -nítido a orçamento de feature fixo.

Destilação entre classes de computação. SMT [Kumar and Isola, 2026] treina uma RNN com um professor paralelo; nossa destilação professor→roteador transfere *roteamento* (não atualizações de estado) e é, ao nosso conhecimento, a primeira recuperação sem rótulos de roteamento de identidade num modelo de memória crescente.

8 Limitações e notas honestas

- Recall sintético de camada única $d = 64$ é o cenário principal; o modelo de linguagem do §5 usa 2 camadas, $d = 128$, TinyStories em nível de caractere, 3 seeds, dois contextos (corpus baixado, sem dados sintéticos). A IGA iguala o baseline de memória fixa em bpc dentro do ruído entre seeds (sem custo de perplexidade), mas o probe de recall de fato plantado é próximo do acaso para ambos nesta escala, e o ganho de $\sim 0,016$ bpc reportado por uma run preliminar de 300 passos *não* reproduz sob o protocolo de 3 seeds 4000 passos; a vantagem de recall associativo é demonstrada em MQAR estruturado (§4), não em recuperação em texto natural em $d = 128$. Escalar d e contexto para surgir uma separação de recall em texto real fica como trabalho futuro (limitado por orçamento de GPU).
- A bijeção indireta é 1-para-1 e determinística; paráfrase real é muitos-para-muitos e ruidosa.
- O braço auxiliar privilegiado usa rótulos de bijeção; o braço de destilação os remove mas adiciona um professor quadrático *apenas durante o treino*. Nenhum braço aprende roteamento

a partir do gradiente de recall sozinho — a análise de família de roteamento do §3.2 sugere que isso é impossível nesse regime (equilíbrio perverso); declaramos como conjectura, não como teorema.

- Mamba é uma reimplementação em PyTorch puro (scan exato, testado por unidade); comparações de wall-clock absolutas a implementações com kernels fundidos seriam injustas e não são feitas — apenas a acurácia da tarefa é comparada.
- O benchmark de custo usa loops Python de referência em ambos os lados (sem kernels fundidos); ele mede *escala*, não constantes de engenharia. O estado recorrente é constante em T (§3.3) e abaixo do cache KV a partir de $T \approx 1k$ em diante; nos menores contextos ($T \lesssim 2K_{\max}S$) o estado constante é dimensionado para capacidade e excede o cache. Segmentos mais antigos que $K_{\max}$ são comprimidos (com perdas), o tradeoff explícito recall/memória.
- Seeds: 3 por célula. Desvio é desvio amostral (Bessel). Nenhuma busca de hiperparâmetro foi feita para nossos modelos além do microbenchmark E0 (p, m, J, r); os baselines receberam a varredura oficial de learning rate.

9 Reprodução

```

pip install -r requirements.txt
python3 tests/test_sanity.py                # 14 equivalence/property tests
                                           # (incl. factored==dense <5e-7, IGA==fixed at init)
python3 scripts/run_cost.py                # inference cost (~5 min)
python3 scripts/run_multiseed.py all       # our models, 3 seeds
python3 scripts/run_baselines_ours.py all  # literature baselines, our protocol
python3 scripts/run_zoology.py all         # official protocol (incl. (512,64))
python3 scripts/aggregate.py               # tables (mean +/- std)
python3 scripts/run_factored_check.py      # factored vs dense recall at (512,64), 3 seeds
python3 scripts/run_fixb_check.py          # capped-memory recall == factored + constant-state table
python3 scripts/project_memory.py          # memory projection (small/medium/1T)
# Real-text language modeling (downloaded TinyStories; isolated folder):
python3 multicamada/run_lm_ml.py --steps 4000 --ctx 1024 --seeds 0,1,2 --tag lm_full
python3 multicamada/run_lm_ml.py --steps 4000 --ctx 2048 --seeds 0,1,2 --tag lm_full_ctx2048

```

O experimento de modelo de linguagem usa o modelo multicamada existente do repositório IGALM (`src/models/iga_layer.py`, $n_{\text{layers}}=2$, $d=128$, factored-capable) comparado contra si mesmo com `gated=False` (o baseline FIXA de largura/parâmetros equivalentes), isolado na pasta `multicamada/`. O corpus é a amostra TinyStories baixada (88 MB); nenhum dado sintético de LM é gerado. Todas as runs são retomáveis (skip-if-exists por JSON em `results/raw/`).

Referências

S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.

- S. Arora, S. Eyuboglu, M. Zhang, A. Timalsina, S. Alberti, D. Zinsley, J. Zou, A. Rudra, and C. Ré. Simple linear attention language models balance the recall-throughput tradeoff. *arXiv preprint arXiv:2402.18668*, 2024.
- A. Behrouz, Z. Li, Y. Deng, P. Zhong, M. Razaviyayn, and V. Mirrokni. Memory caching: RNNs with growing memory. *arXiv preprint arXiv:2602.24281*, 2026.
- M. Charikar, K. Chen, and M. Farach-Colton. Finding frequent items in data streams. In *ICALP*, 2002.
- K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlós, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller. Rethinking attention with performers. In *ICLR*, 2021.
- G. Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- P. Kacham, V. Mirrokni, and P. Zhong. PolySketchFormer: Fast transformers via sketching polynomial kernels. *arXiv preprint arXiv:2310.01655*, 2023.
- A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.
- A. Kumar and P. Isola. Pretraining recurrent networks without recurrence. *arXiv preprint arXiv:2606.06479*, 2026.
- N. Pham and R. Pagh. Fast and scalable polynomial kernels via explicit feature maps. In *KDD*, 2013.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.